



Python Lists – Lecture Notes

Course: FastAPI - The Complete Course 2026 (Beginner + Advanced)

Lecture 22: Lists in Python (~9 min)

What is a List?

A list is a collection of data stored in a single variable. Unlike a regular variable that holds one value, a list holds **multiple values**.

Lists are created with **square brackets** `[]`, with elements separated by commas.

```
my_list = [80, 96, 72, 100, 8]
print(my_list) # [80, 96, 72, 100, 8]
```

Lists can hold any type — integers, strings, etc.

```
people_list = ["Eric", "Adele", "Jeff"]
print(people_list) # ['Eric', 'Adele', 'Jeff']
```

Indexing

Every element in a list has an **index starting at 0**.

INDEX	0	1	2	3	4
Value	80	96	72	100	8

```
print(my_list[0]) # 80
print(my_list[4]) # 8
print(my_list[5]) # ❌ IndexError: list index out of range
```

Negative indexing: `-1` always grabs the **last** element.

```
print(people_list[-1]) # "Jeff"
```

Reassigning by index:

```
people_list[0] = "Mel"
print(people_list[0]) # "Mel"
```

Length

Use `len()` to get the number of elements.

```
print(len(people_list)) # 3
```

The length is 3 even though the highest index is 2.

Slicing

Slicing extracts a subset of the list using the syntax `[start:stop]`. The **start** index is included, the **stop** index is **not**.

```
print(people_list[0:2]) # ['Eric', 'Adele'] - stops before index 2
```

```
print(my_list[0:2])      # [80, 96]
```

```
print(my_list[0:4])      # [80, 96, 72, 100] - stops before index 4
```

```
print(my_list[2:4])      # [72, 100] - starts at index 2
```

Inserting & Deleting

`append()` — add to the end

```
my_list.append(1000)
print(my_list) # [80, 96, 72, 100, 8, 1000]
```

`insert(index, value)` — add at a specific position

```
my_list.insert(2, 1000)
print(my_list) # [80, 96, 1000, 72, 100, 8, 1000]
```

All elements from that index onward shift to the right.

`remove(value)` — delete by value

```
my_list.remove(8)
# Removes the element whose value is 8
```

`pop(index)` — delete by index

```
my_list.pop(0)
# Removes the element at index 0 (i.e. 80)
```

Sorting

`sort()` arranges numeric values from least to greatest (in-place).

```
my_list.sort()
print(my_list) # [72, 96, 100, 1000, 1000]
```

Key Takeaways

1. **Indexing starts at 0** — the most common beginner mistake.
2. **Negative index** `-1` is a shortcut to the last element.
3. **Slicing** `[start:stop]` includes `start` but **excludes** `stop`.
4. `remove()` **matches by value**, `pop()` **matches by index** — know the difference.
5. `append()` adds to the end; `insert()` adds at any position.
6. `sort()` mutates the list in place.
7. Lists are used constantly in real development — this is a foundational concept.